```
#!/usr/bin/env python3
"""
.uni Format Reference Engine (v8.2 — Pure Binary Zero-JSON & Zero-Copy Architecture)
Eliminates all legacy text/JSON serialization inside operational nodes.
All Knowledge Graph triplets (0x03) are 32-byte AVX2/NEON aligned (<QQQeeI). """ #
(<32s32s64sQIB) (<IBBHI32s) (<If) (<Nf) (<QQQeeI) (<ffffHH) (ZERO (only * + - 1. 141B 20B
32-BYTE 32B 3D 48B 4B 754 8B 92% ALIGNED) Any BEHAVIOR_CONTRACT="0x05"
BINARY DMA Dict, Exhaust GENESIS_TEXT="0x06" HOLO_WAVEFRONT="0x07"
HVACMode(IntEnum): Holo-Frame IEEE IR/UV IntEnum, IntFlag JSON! JSON,
KNOWLEDGE_GRAPH="0x03" Legacy List, N NODE NOTARY_ANCHOR="0x01" NPU
PASSIVE_IDLE="0" PURE Pockels/MEMS ProtoFlags(IntFlag):
RS_FEC_APPENDED="0x0002" SCHEMAS SEMANTIC_TENSOR="0x02"
SOLAR_SHIELD_SUMMER="1" SPATIAL_FRUSTUM_BROADCAST="0x0008"
STREAM_DELTA_FRAME="0x0004" Supports TAMPER_MESH_INTACT="0x0001"
TELEMETRY_MATRIX="0x04" THERMAL_TRAP_WINTER="2" Tuple, UTF-8
UniNodeType(IntEnum): VENTURI_HVAC_RECOVERY="3" VERSION_8_2="0x0820" Venturi
Waveguide ZERO Zero-Copy aligned and binary building class contract controllers. coupled def
direct enum export) external fallback fixed float32 for from hashlib heat heating hmac import
intake math memoryview metric_id old-world photon reflection retention
semantic_id64(concept_str: slicing str) struct to triplets typing vectors venting voxel wind —> int:
    """Converts any human word/concept into a deterministic 8-byte (64-bit) semantic ID."""
    digest = hashlib.blake2b(concept_str.encode("utf-8"), digest_size=8).digest()
    return struct.unpack("<Q", """ # & (0x03): (0x05): (0x07): (20 (32 (48 (4B (AVX2 + - / 2. 20 32
48 4B 8 <IBBHI32s <QQQeeI <ffffHH @classmethod ARM Any]]) BINARY
CONTRACT_FMT="<IBBHI32s" Contract ENGINE Ego_f16, Flags_4B) HVACMode,
LaserCap_mW, List[Dict[str, MEMORYVIEW MediaHash_32B) NEON Obj_8B, Phase_2pi,
Pred_8B, Pulse_mW) PureBinaryEngine: RGB565, Replaces SERIALIZER Sensor_Hash
Subj_8B, TELEMETRY_FMT="<If" TRIPLET_FMT="<QQQeeI" TTL, Triplet UI_Intent,
VOXEL_FMT="<ffffHH" Wavefront Weight_f16, X, Y, Z, ZERO-COPY Zone, aligned) binary
bytes cache-line class def digest)[0] exact: fixed float32 pack_knowledge_graph_binary(cls,
parsing structs: text triplets: value) with> bytes:
        buf = bytearray(len(triplets) * 32)
        offset = 0
        for t in triplets:
            s_id = t["s"] if isinstance(t["s"], int) else semantic_id64(t["s"])
            p_id = t["p"] if isinstance(t["p"], int) else semantic_id64(t["p"])
            o_id = t["o"] if isinstance(t["o"], int) else semantic_id64(t["o"])
            flags = 1 if t.get("is_delta_conflict", False) else 0
            struct.pack_into(
                cls.TRIPLET_FMT, buf, offset,
                s_id, p_id, o_id,
                float(t.get("weight", 1.0)),
                float(t.get("ego_alignment", 0.0)),
                flags
```

```
            )
            offset += 32
        return bytes(buf)

    @classmethod
    def zero_copy_iter_triplets(cls, raw_view: memoryview):
        """
        Zero-Copy generator: iterates directly over mapped RAM (memoryview)
        without allocating a full list or parsing strings.
        """
        for s_id, p_id, o_id, weight, ego, flags in struct.iter_unpack(cls.TRIPLET_FMT, raw_view):
            yield {
                "s_id64": hex(s_id),
                "p_id64": hex(p_id),
                "o_id64": hex(o_id),
                "weight": round(weight, 4),
                "ego_alignment": round(ego, 4),
                "is_delta_conflict": bool(flags & 1),
            }

    @classmethod
    def merge_binary_graphs_in_ram(cls, view_a: memoryview, view_b: memoryview) -> bytes:
        """
        Hardware-style binary Knowledge Graph Triage (Zone 2):
        Compares 64-bit integer IDs directly in RAM and flags Delta Conflicts
        in superposition without a single string comparison!
        """
        grouped: Dict[Tuple[int, int], List[Tuple[int, float, float]]] = {}
        for view in (view_a, view_b):
            for s_id, p_id, o_id, w, ego, _ in struct.iter_unpack(cls.TRIPLET_FMT, view):
                key = (s_id, p_id)
                grouped.setdefault(key, [])
                if not any(existing[0] == o_id for existing in grouped[key]):
                    grouped[key].append((o_id, w, ego))

        out_buf = bytearray()
        for (s_id, p_id), edges in grouped.items():
            conflict_flag = 1 if len(edges) > 1 else 0
            for o_id, w, ego in edges:
                out_buf.extend(struct.pack(cls.TRIPLET_FMT, s_id, p_id, o_id, w, ego, conflict_flag))
        return bytes(out_buf)

    @classmethod
    def pack_behavior_contract_binary(cls, contract: Dict[str, Any]) -> bytes:
```

```
        media_hash = bytes.fromhex(contract.get("media_sha256_hex", "00" * 32))
        return struct.pack(
            cls.CONTRACT_FMT,
            int(contract.get("ttl_sec", 3600)),
            int(contract.get("zone", 1)),
            int(contract.get("hvac_mode", HVACMode.SOLAR_SHIELD_SUMMER)),
            int(contract.get("ui_intent_code", 0x0001)),
            int(contract.get("laser_pulse_cap_mw", 850)),
            media_hash,
        )

    @classmethod
    def unpack_behavior_contract_binary(cls, raw_view: memoryview) -> Dict[str, Any]:
        ttl, zone, hvac, ui_code, mw, m_hash = struct.unpack(cls.CONTRACT_FMT, raw_view)
        return {
            "ttl_sec": ttl,
            "zone": zone,
            "hvac_mode": HVACMode(hvac).name,
            "ui_intent_code": hex(ui_code),
            "laser_pulse_cap_mw": mw,
            "media_sha256_hex": m_hash.hex(),
        }


# ==============================================================================
# 3. VERIFICATION OF ZERO-COPY & ZERO-JSON EXECUTION
# ==============================================================================
if __name__ == "__main__":

print("==============================================================================")
    print(" .uni v8.2 Pure Binary Zero-Copy & Zero-JSON Verification")

print("==============================================================================\n"
)

    # 1. Pack Knowledge Graph into 32-byte aligned binary registers (No JSON!)
    graph_a = PureBinaryEngine.pack_knowledge_graph_binary([
        {"s": "Holo-Frame", "p": "deflects_via", "o": "Pockels_LiNbO3", "weight": 0.99,
"ego_alignment": 0.95},
        {"s": "Dump_Valve", "p": "triggers_at", "o": "1.2_atm", "weight": 0.95, "ego_alignment":
0.90},
    ])
    graph_b = PureBinaryEngine.pack_knowledge_graph_binary([
```

```
        {"s": "Dump_Valve", "p": "triggers_at", "o": "1.5_atm", "weight": 0.80, "ego_alignment":
0.85},
    ])

    print(f"[1] Binary Knowledge Graph A Size : {len(graph_a)} bytes (Exactly 32B per triplet)")

    # 2. Zero-Copy RAM Merge via memoryview (No string parsing!)
    merged_ram_bytes = PureBinaryEngine.merge_binary_graphs_in_ram(
        memoryview(graph_a), memoryview(graph_b)
    )
    print(f"[2] Merged Graph in RAM           : {len(merged_ram_bytes)} bytes (3 triplets)")
    for idx, item in
enumerate(PureBinaryEngine.zero_copy_iter_triplets(memoryview(merged_ram_bytes))):
        conflict_str = " [SUPERPOSITION CONFLICT]" if item["is_delta_conflict"] else ""
        print(f"    Triplet #{idx+1}: {item['s_id64']} -> {item['p_id64']} -> {item['o_id64']}{conflict_str}")

    # 3. Pack 48-byte Binary Behavior & HVAC Contract
    bin_contract = PureBinaryEngine.pack_behavior_contract_binary({
        "ttl_sec": 60,
        "zone": 1,
        "hvac_mode": HVACMode.SOLAR_SHIELD_SUMMER,
        "ui_intent_code": 0x0001,
        "laser_pulse_cap_mw": 850,
    })
    decoded_contract =
PureBinaryEngine.unpack_behavior_contract_binary(memoryview(bin_contract))
    print(f"\n[3] Binary Behavior Contract Size : {len(bin_contract)} bytes (Fixed 48B struct)")
    print(f"    Zero-Copy Readout             : {decoded_contract}")
```